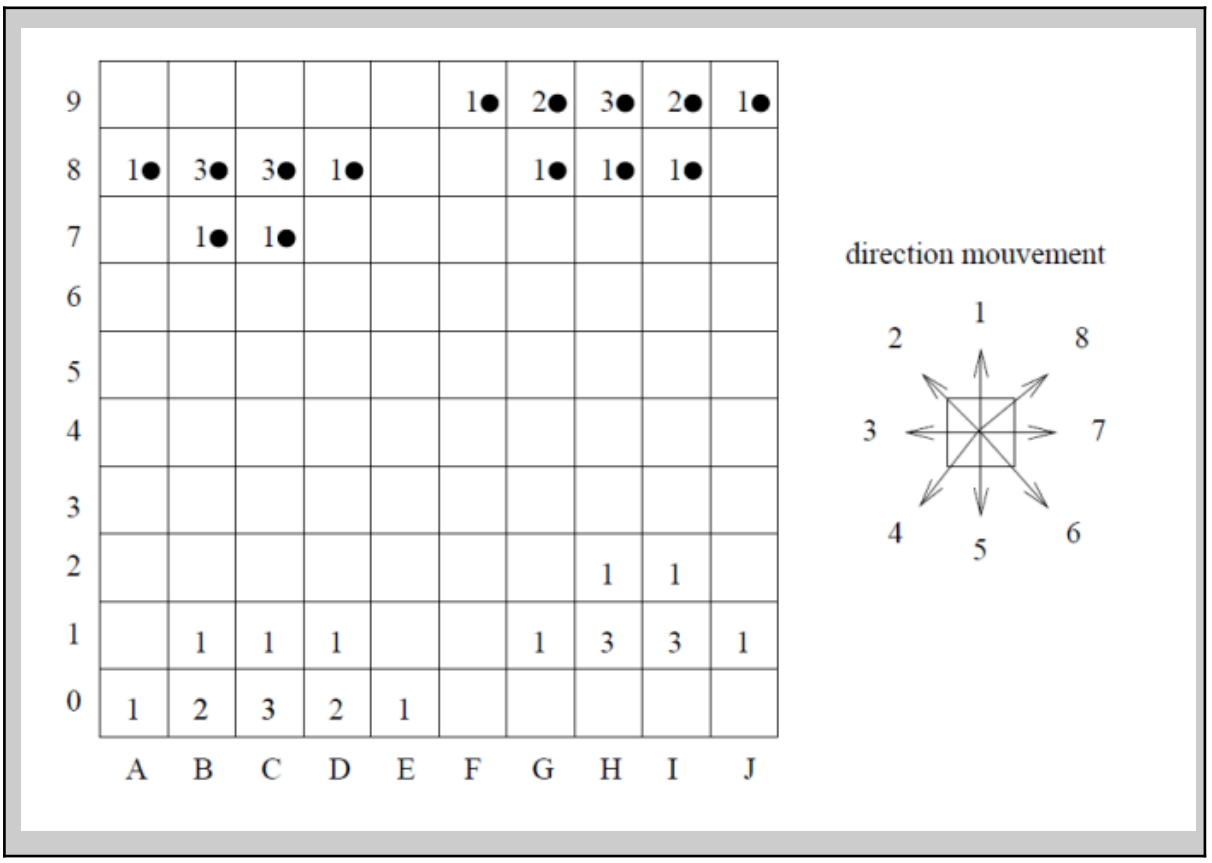
1st TD Report : Wargame

Intelligence Artificielle



Student : Omar HACHANI
Student ID: 22301138
Report's deadline : October 25th 2026

1. Choices

Before any AI optimizations or strategy making, I have decided that I should start by making this program functional, because I have not practiced C in a while. That's why I spent the first two weeks analyzing and trying to understand it (That was when I saw with my colleague Dania Chabane that an attacking pawn's value did not count in an attack). In the third week, I started coding and pushing my code in its repo on Github, where I implemented rudimentary functions and strategies only to make this work together.

I chose to implement Minimax and Alpha-beta in different C files, because only a few lines of codes will change. I will start by fully implementing the Minimax algorithm and try to improve my strategy with it. I will then implement Alpha-beta only after finishing my strategy, because it should not be giving me better "choices", it'll only help with complexity.

2. Implementation

I will use italic and blue color for *functions*, and red for variables or values.

I started by implementing the Minimax algorithm through *f_min* and *f_max*. Both functions explore possible moves, but they represent different players: *f_max* keeps the highest score for the AI, while *f_min* keeps the lowest score, representing the opponent's best response. The joueur parameter always keeps the AI's colour, so all positions are evaluated from the same point of view. When the depth reaches zero, the functions return the score calculated by *f_eval*.

For each piece, the program tries the neighbouring cases and checks whether the movement is allowed with *f_test_mouvement*. Each valid move is simulated on a **copy** of the board, because *f_bataille* uses the global variable plateauDeJeu (the same goes for *f_bouge_piece*). To make the combat calculation use the correct position, I saved the global pointer, temporarily redirected it to the copy, and restored it after the simulation.

```
if(jeu[l*NB_COLONNES+c].couleur == joueur)
{
    for(dl=-1; dl<=1; dl++)
    {
        for(dc=-1; dc<=1; dc++)
        {
            if(f_test_mouvement(jeu, l, c, l+dl, c+dc, joueur) == 0)
            {
                f_copie_plateau(jeu, copie);
                sauvegarde = plateauDeJeu;
                plateauDeJeu = copie;
                f_bouge_piece(copie, l, c, l+dl, c+dc, joueur);
                plateauDeJeu = sauvegarde;
                score = f_min(copie, profondeur-1, joueur);
                if(score > maxi)
                    maxi = score;
            }
```

For the evaluation function *f_eval*, I made it combine the difference in strength between both armies with their progress towards the opposite side. The strength difference is multiplied by 10, while each piece receives a bonus equal to the square of its advancement (for example, an 'o' pawn, going from bottom to top, will have a bonus of 15 when going from line 7 to line 8, because his bonus will be going from 7^2 to 8^2). My intention was to encourage forward movement, especially for pieces already close to the opponent's back row. These coefficients were arbitrary choices whose effects could be optimised.

```
score = 10 * (f_valeur(jeu, joueur) - f_valeur(jeu, -joueur));
```

```
// player 1 'x' starts from the last line and tries to reach line 0
if(jeu[l*NB_COLONNES+c].couleur == 1)
{
    avance = NB_LIGNES-1-l;
    // using the square so that pawns closer to victory count more
    avancement1 += avance * avance;
}
// player -1 'o' moves in the opposite direction
else if(jeu[l*NB_COLONNES+c].couleur == -1)
{
    avance = l;
    avancement2 += avance * avance;
}
```

I also thought about a victory check before the depth limit in both Minimax functions. If a simulated position ends the game, the search stops on that branch and returns INFINI for an AI victory or -INFINI for a defeat. This gives an actual victory priority over an ordinary advantage in strength or position. We can see this in *f_max* :

```
if(gagnant == joueur)
    return INFINI;
if(gagnant == -joueur)
    return -INFINI;

if(profondeur <= 0)
    return f_eval(jeu, joueur);

for(l=0; l<NB_LIGNES; l++)
{
```

Finally, *f_IA* evaluates the available moves and saves the coordinates of the best one. The total search depth is fixed at 3: the move simulated in *f_IA* counts as the first move, so *f_min* receives a remaining depth of 2. The first valid move is accepted even if its score is -INFINI, ensuring that the AI still plays when every option is losing. Immediate victories are preferred when scores are equal. After examining the moves, the program plays only the selected one on the real board.

3. Algorithm performances

For now, the algorithms' efficiency can be measured only by time and the number of nodes explored. So I started every algorithm and measured the time they took to finish the game, and implemented a counter for the number of nodes explored in them.

This game being deterministic, we know that we'll find the same solution using both algorithms. Which means that the stats we have are very useful comparing them.

Algorithm	**Number of rounds**	**Counter after first move**	**Total of nodes**	**Time taken**
Minimax	70	81 732	14 077 826	43 s
Alpha-beta	70	2 985	1 716 569	9 s

We can see that alpha-beta is significantly more efficient than Minimax. It explores **1,716,569** nodes, whereas Minimax explores **14,077,826** nodes. This represents a reduction of approximately **89%** in the number of explored nodes. The execution time is also reduced, from 42 seconds with Minimax to 9 seconds with Alpha-beta, which makes Alpha-beta approximately **4.7** times faster in this test.

As expected, Alpha-beta does not change the result of the game, but avoids exploring branches that cannot improve the final decision. Its advantage becomes particularly important when the search depth increases, because the number of possible positions grows quickly.